```
const express = require('express');
const http = require('http');
const crypto = require('crypto');
const nodemailer = require('nodemailer');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);
app.use(express.json());
app.get('/', (req, res) => res.sendFile(__dirname + '/index.html'));

const DOMAIN = '@vitstudent.ac.in';
const HOURLY_LIMIT = +(process.env.HOURLY_LIMIT || 5); // reports in 1 hour -> 24
const REPORT_LIMIT = +(process.env.REPORT_LIMIT || 30); // reports before permane
const sha = s => crypto.createHash('sha256').update(s).digest('hex');
const mailer = process.env.SMTP_HOST ? nodemailer.createTransport({
  host: process.env.SMTP_HOST, port: +(process.env.SMTP_PORT || 587),
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
  connectionTimeout: 10000, greetingTimeout: 10000, socketTimeout: 15000
}) : null;

const otps = new Map(), tokens = new Map(), reportsBy = new Map(), banned = new S
const HOUR = 3600000;
const blocked = h => banned.has(h) || (tempBan.get(h) || 0) > Date.now();

// ---------- VIT email verification ----------
app.post('/api/otp', async (req, res) => {
  const email = String(req.body.email || '').trim().toLowerCase();
  if (!/^[a-z0-9._-]+@vitstudent\.ac\.in$/.test(email)) return res.status(400).js
  if (blocked(sha(email))) return res.status(403).json({ e: 'Your account is curr
  const old = otps.get(email);
  if (old && Date.now() - old.sent < 60000) return res.status(429).json({ e: 'Ple
  const code = String(crypto.randomInt(100000, 999999));
  otps.set(email, { code, exp: Date.now() + 10 * 60000, tries: 0, sent: Date.now(
  try {
    if (mailer) await mailer.sendMail({ from: process.env.SMTP_FROM || process.en
      subject: 'Your Orbit verification code: ' + code, text: `Your verification
    else if (process.env.RENDER) { console.error('SMTP not configured: set SMTP_H
    else console.log('DEV OTP for', email, code);
    res.json({ ok: true });
  } catch (e) { console.error('MAIL ERROR:', e.message); res.status(500).json({ e
});
```

```
app.post('/api/verify', (req, res) => {
  const email = String(req.body.email || '').trim().toLowerCase();
  const o = otps.get(email);
  if (!o || Date.now() > o.exp || ++o.tries > 5) return res.status(400).json({ e:
  if (o.code !== String(req.body.code)) return res.status(400).json({ e: 'Wrong c
  otps.delete(email);
  const token = crypto.randomBytes(24).toString('hex');
  tokens.set(token, { email, h: sha(email) });
  res.json({ token });
});

// ---------- matching ----------
const A = ['Silent', 'Chill', 'Sleepy', 'Happy', 'Brave', 'Funky', 'Sassy', 'Cool
const B = ['Panda', 'Tiger', 'Maggi', 'Samosa', 'Owl', 'Coder', 'Topper', 'Backbe
const nick = () => A[Math.floor(Math.random() * A.length)] + ' ' + B[Math.floor(M

const waiting = [], partner = new Map(), users = new Map();
const compat = (a, b) => (a.want === 'any' || a.want === b.gender) && (b.want ===
const shared = (a, b) => a.interests.filter(i => b.interests.includes(i)).length;

function unpair(id, notify = true) {
  const p = partner.get(id);
  if (p) { partner.delete(id); partner.delete(p); if (notify) io.to(p).emit('part
  const i = waiting.findIndex(w => w.id === id);
  if (i > -1) waiting.splice(i, 1);
}
function find(sock, prefs) {
  unpair(sock.id);
  const u = users.get(sock.id); u.reveal = false;
  const me = { id: sock.id, ...prefs };
  let best = -1, score = -1;
  waiting.forEach((w, i) => { if (w.id !== me.id && compat(me, w)) { const s = sh
  if (best === -1) { waiting.push(me); return sock.emit('searching'); }
  const o = waiting.splice(best, 1)[0];
  partner.set(me.id, o.id); partner.set(o.id, me.id);
  users.get(o.id).reveal = false;
  const n1 = nick(), n2 = nick(), common = me.interests.filter(i => o.interests.i
  io.to(o.id).emit('matched', { initiator: true, nick: n1, common });
  sock.emit('matched', { initiator: false, nick: n2, common });
}

io.use((s, next) => {
  const t = tokens.get(s.handshake.auth && s.handshake.auth.token);
  if (!t) return next(new Error('auth'));
  if (blocked(t.h)) return next(new Error('banned'));
  s.user = t; next();
});
```

```
io.on('connection', sock => {
  users.set(sock.id, { ...sock.user, reveal: false });
  io.emit('online', io.engine.clientsCount);
  const peer = () => partner.get(sock.id);

  sock.on('find', p => {
    if (!['boy', 'girl'].includes(p.gender) || !['boy', 'girl', 'any'].includes(p
    const interests = (Array.isArray(p.interests) ? p.interests : []).slice(0, 5)
    find(sock, { gender: p.gender, want: p.want, mode: p.mode, interests });
  });
  sock.on('signal', d => { const p = peer(); if (p) io.to(p).emit('signal', d); }
  sock.on('msg', t => { const p = peer(); if (p && typeof t === 'string') io.to(p
  sock.on('typing', () => { const p = peer(); if (p) io.to(p).emit('typing'); });
  sock.on('leave', () => unpair(sock.id));

  // mutual consent: both tap "Let's meet" -> each sees the other's VIT email
  sock.on('reveal', () => {
    const p = peer(); if (!p) return;
    users.get(sock.id).reveal = true;
    io.to(p).emit('reveal-asked');
    if (users.get(p).reveal) {
      io.to(sock.id).emit('revealed', users.get(p).email);
      io.to(p).emit('revealed', users.get(sock.id).email);
    }
  });
  sock.on('report', () => {
    const p = peer(); if (!p) return;
    const t = users.get(p);
    if (t) {
      const r = reportsBy.get(t.h) || { total: 0, recent: [] };
      const now = Date.now();
      r.total++; r.recent = r.recent.filter(x => now - x < HOUR); r.recent.push(n
      reportsBy.set(t.h, r);
      console.log('REPORT', t.h.slice(0, 8), 'total', r.total, 'lastHour', r.rece
      let perm = null;
      if (r.total >= REPORT_LIMIT) { banned.add(t.h); perm = true; }
      else if (r.recent.length >= HOURLY_LIMIT) { tempBan.set(t.h, now + 24 * HOU
      if (perm !== null) { io.to(p).emit('banned', { perm }); io.sockets.sockets.
    }
    unpair(sock.id);
  });
  sock.on('disconnect', () => { unpair(sock.id); users.delete(sock.id); io.emit('
});

server.listen(process.env.PORT || 3000, () => console.log('Running'));
```